Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous research draft

Abstract

AI-generated mathematics changes what proof verification asks the machine to do. A mathematical witness can be checked without repeating its producer's search, while a computational certificate can require the kernel to recompute the calculation encoded in the proof. We characterize these two cost regimes using released `openai/math` artifacts and their version-matched Mathlib dependency. The same comment/string-masked scanner finds 1,354,379 `decide +kernel` sites in 163,097 generated-library files, versus 7 sites in 8,529 Mathlib files: approximately 10,668.3 times the source-megabyte density. Only 5,848 generated-library files (3.5856%) contain this form. This is lexical concentration, not a measured fraction of verification time. Runtime-weighted prevalence remains [**RESULT: pending from the incomplete native sample**]. We examine repeated imports and production/checking coupling, and test single-process streaming and automatically chunked original-kernel checking as design-implication prototypes. Four selected matched closure checks cost 6.76–14.55% of serial source compiler work. These aggregates hide costly recomputation within computational targets. A dependency-aware prototype reduces independent replay wall time 3.33-fold on one compiled module, with 27.8% more native CPU. The meaning of *verified* is a separate boundary: the default build omits 33.30% of kernel-decision sites, outcomes depend on settings and observation, and whole-library axiom closure remains unestablished. The contribution is a characterization and its tested implications, not a general accelerator or a replacement trusted kernel.

1 Introduction

Checking a proof need not resemble producing it. Mathematical proof production can involve search, tactics and elaboration; an independent kernel can instead check the supplied witness. A computational proof exposes a different boundary: when its justification depends on reducing a large decidable expression, checking executes the relevant computation. Replaying the unchanged certificate can repeat that work.

Our primary workload is the released `openai/math` research library [21]. Its pinned Mathlib dependency supplies a version-matched reference [7]. The generated release contains far more source-level kernel evaluation, concentrated in a small subset of files. This motivates a distinction between mathematical-witness checking and proof-embedded recomputation, rather than treating verification as one uniform solver call. Files can mix both regimes; tokens cannot assign CPU.

The distinction suggests two systems responses. For mathematical proofs, can checking avoid producer elaboration and repeated import preparation? For computation, can independent claims be checked by several unchanged kernels and assembled into the original theorem? A faster untrusted numeric answer, weaker import trust scope or new axiom is not equivalent verification. We organize the study around four questions:

RQ1 Where do verification costs fall, and how prevalent are the computation-replaying and mathematical-proof-checking regimes across the library?

RQ2 Why does independent verification fail to realize the potential low cost of mathematical-proof checking?

RQ3 Can computational checking scale across cores or a cluster without expanding the trusted base?

RQ4 What gaps remain between a reported verified result and build coverage, reproducible outcomes and known axiom dependencies?

Each RQ separates evidence, missing results and design implications. We use public generated-software outputs and retained candidate streams as contrasts, never generate replacements for unavailable releases. Streaming and chunking are prototypes testing the characterization; warm imports, ordinary scheduling and external search/checking are prior techniques. Missing native profiles and full-library weights remain explicit.

2 Methodology

2.1 Sources and the common lexical boundary

The primary release is `4eaee87548250b2682affda7d05d947c05554bae`, with Lean 4.34.1 and Mathlib

d13f23b723b8a846827a245b89c10fc7d3f11612. We compare all released Lean files in the primary inventory with the reference repository's Mathlib/ subtree, excluding tests, archive and tooling directories from that reference denominator. The reference source tree is checked at its exact Git revision. All 163,097 primary source members are independently rescanned from the pinned official archive and matched by path/byte size to the complete inventory; decoding and lexical checks succeed. The local extracted cache is partial and is not used as the population. A complete received-archive-prefix scan and full-archive scan agree exactly; they are repeated validation, not additional corpus observations. The same existing scanner erases nested comments and string literals and matches decide, whitespace, +, whitespace and kernel, allowing zero whitespace. One MB is 1,000,000 original UTF-8 source bytes. Sites and containing files are not dynamic calls, distinct propositions, accepted proofs or native time.

Vericoding releases 1,503 model-tagged Lean snapshots, 1,490 distinct source texts; its original 626 are a subset [31]. Publisher labels are not our native verdicts or axiom audit. The inspected Vero release does not establish availability of evaluated agent outputs [33]; scaffolds/references do not replace them. Lean Kernel Verify supplies checking code and KIR fixtures with separate output provenance [20]. **[RESULT: Version-qualified generated-software native costs and axiom audit.]** The contrast streams retain 7,808 miniF2F, 5,952 ProofNet per recorded pass and 3,566 AutoVerus sources, including unsuccessful attempts. Source-sorted replay is not historical arrival traffic.

2.2 Cost boundaries and regime classification

Source production includes imports, elaboration, tactics, proof construction, first checking and requested artifact emission. AI inference time is excluded unless separately supplied by publisher logs. Independent checking consumes compiled declarations under a declared trust scope. A kernel-only subclock omits preparation/cleanup and cannot replace either complete process clock.

The regimes describe checking work, not disjoint proof languages. Computational checking normalizes embedded calculations; mathematical checking can avoid the producer's search and elaboration. A module may contain both. A C++ checker sample includes inference, equality, normalization and cleanup, not solely final validation. User-cycle percentages are not elapsed-time shares. **[RESULT: Representative no-memory-option source-compilation phase profiles, classification error, observer uncertainty and runtime-weighted prevalence.]**

2.3 Sampling and current native point

The original frame remains 163,097 modules, with 21 nonempty strata formed from four source-size bins, three internal-import-depth bins and masked kernel-decision presence. Seed 20261030 selects two draws per stratum, 42 total. Inclusion probability is $2/N_h$; purposeful modules and prerequisites receive no additional population weight. For complete strata,

$$\widehat{T}_h = N_h \bar{y}_h, \qquad \widehat{V}_h = N_h^2 (1 - n_h/N_h) s_h^2 / n_h, \quad n_h = 2.$$

Missing draws stay missing, not replaced by completed-response counts. Two-draw normal-cost intervals are fragile under heavy tails. The retained W no-`-M` source point has 31 successful native calls: 29 original random draws and two purposeful modules. All original sources, no-`-M` arguments, PID/birth retirement and matching native clock sums are checked. The random-frame coverage is 29/42 with 12 complete strata; 13 draws remain missing. The two purposeful modules receive no sampling weight. These are local compilation costs with prerequisite objects available, not cold transitive builds, whole Lake execution or generation time. **[RESULT: Complete no-memory-option sample and per-regime cost estimates.]**

The platform is phobos-vt, Intel Core i9-14900K, observed 64,205,640 KiB RAM. Timed native captures share one P0–7 lock; preparation/analysis use E cores. Eight requested workers are not assumed to use eight CPU cores. Current measurements omit `-M`, including `-MO`. The required memory-protection boundary is external cgroup; a result records its actual enforcement rather than assuming conformance. The W point uses 15 GiB charged native memory and separate one-second sampled native/controller RSS response. Charged pages, leader RSS and tree RSS differ; sampled response can overshoot and is not a kernel hard aggregate-RSS quota. Every result must record its actual enforcement boundary.

A measurement correction, not a workload finding. The pinned runtime initializes its native memory limit to zero; `check_memory` returns before RSS queries while zero [14]. Our earlier optional `-M` setting introduced periodic accounting. Three ordered plain repetitions per arm give guarded/no-`-M` wall-median ratios 2.03/1.61/1.01 for Grid/UMoments9/EnvelopePsiB, with matching verdicts, diagnostics and emitted artifacts. The same outer 15 GiB cgroup remains; output and heartbeat settings match within each module. These passing cases are not resource-failure equivalence. The earlier five-source `openai/math` panel's 75–94% versus roughly 2% user-cycle contrast, and a 13-short-module 1.28 replay/build ratio, also used `-M` builds. The panel is not a Mathlib native sample. They motivate questions, not current runtime conclusions. Affected costs/projections remain archived, never scaled

by Grid's ratio.

2.4 Outcomes, observation and trust

Process wall, user/system CPU, queue/capture lifecycle and extra rechecks keep their own denominators. Unprofiled, sampled and traced executions are separate conditions. Ordinary error, timeout, cgroup OOM, watchdog interruption, platform loss and never-started dependency are distinct; none proves falsehood. Raw, failed and partial captures remain retained on the PVC.

Import-relative stock `leanchecker` trusts imported declarations. Its `--fresh` mode replays the loaded constant map into an empty kernel. Both use the upstream kernel and designated unsafe/partial exclusions [16]; neither is an independent implementation or an axiom-closure audit. A cheaper trust scope cannot substitute for equivalence.

3 Kernel-evaluation forms are concentrated; runtime weights remain open

RQ1: checking versus recomputing. Table 1 gives the complete, common-boundary source comparison. Unmasked Mathlib text contains eight sites in five files. One is a documentation comment in `Mathlib/Tactic/Echelon/Cert.lean`. Removing comments gives seven in four files; the table uses this common masked boundary, not eight unmasked sites against masked generated code.

The density ratio is 10,668.3, approximately 1.1×10^4. Only 3.5856% of generated-library files contain the form. These are concrete candidates for inspecting recomputation, not an exhaustive runtime classification: expensive reduction can occur without the token, and a positive file can mix computation and symbolic reasoning. The 29 completed no-`-M` random targets sum to 367.024 seconds of native wall intervals and 602.628 user-plus-system CPU seconds. This is not a full-build makespan or population expansion. The separate purposeful Grid and Amplitudes points differ in output/heartbeat scope and do not fill missing strata. Unprofiled process costs do not supply a kernel-cycle partition. A source-defined proxy marks files containing a masked explicit kernel-decision form; it does not establish which dynamic phase dominates them. For the 12 completed strata only (101,596 modules), the stratified estimates are 15.468 CPU hours in marked files and 116.705 hours in other files, giving a conditional marked-file share of 11.70%. These are compilation CPU estimates, including imports, tactics and emission, not kernel-check-only hours or whole-library totals. The remaining strata contain 61,501 modules, including 3,036 of the 5,848 marked files. Over half the marked population is outside this estimate. Two draws per stratum leave substantial uncertainty: a nominal joint Bonferroni interval spans 0–100%, even before missing-stratum or label error. [**RESULT: Representative no-`-M` phase profiles and full-library regime-specific cost weights after completing the original sample.**] The 3.5856% source-file concentration must not become a cost fraction.

Native checking work has different useful granularities. A separate completed capture replays the 31 retained compiled targets: 29 available probability-sample artifacts and two purposeful modules. It contains 34 successful native target executions, including stock/timed pairs for UMoments9, BarrierData41 and Grid. Each target trusts its declared imported environment and replays its own eligible declarations; this is not full-closure checking or source compilation. A wrapper around the original C++ checking call records per-call thread CPU. Decision-shaped theorem bodies have `of_decide_eq_true` at their head after annotation cleanup. This syntactic proof-term class does not exhaust computational proofs; it is defined by stored terms rather than the source tactic name `native_decide`.

Grid's 4,608 such declarations cost 263.619 CPU seconds in aggregate, but none exceeds 186 ms individually (Table 2). UMoments9 and BarrierData41 similarly contain many subsecond calls. EnvelopePsiB instead has ten calls above one CPU second, totaling 55.782 of its 58.006 seconds (96.17%). These require different decomposition opportunities despite both carrying kernel-decision forms.

Across the unweighted panel, all native checking calls consume 588.773 thread-CPU seconds; this decision-shaped class consumes 574.744 seconds. Its twelve calls above one second consume 59.402 seconds: 10.09% of all checking-call CPU, or 10.34% of this class's CPU. Weighting only the 29 available probability-sample artifacts gives 28.23% and 34.93%, respectively. These are conditional available-case ratios, not unbiased full-library estimates or upper bounds on parallelizable work: thirteen original draws are absent and nonresponse is not corrected. They must not be multiplied into source-compiler or whole-closure costs.

The stock/timed acceptance sets agree for all three paired modules. Their single ordered timed/stock replay-CPU ratios are 0.9100/1.0016/1.0745. Formatting occurs outside each native-call timer, but allocation/cache and observer effects can still change the experiment. This is not a zero-overhead or unperturbed source-profile claim. The capture records P0–7, no `-M`, an external 14 GiB charged-memory cap and a separate sampled RSS response. All 34 target processes and their cgroups, plus the controller, are retired.

Checking an unchanged computation-based certificate repeats the reductions needed by its justification. It need not repeat the producer's search, but cannot treat an untrusted numerical answer as a theorem. For a math-

Corpus	Lean files	Source MB	Kernel-decision sites	Containing files	Sites/MB
Mathlib `d13f`	8,529	98.450	7	4	0.0711
`openai/math`	163,097	1785.499	1,354,379	5,848	758.544

Table 1: Nested-comment/string-masked `decide +kernel` sites. Decimal MB uses original source bytes. Lexical counts are not time weights.

Module	Calls	CPU (s)	Max (ms)	$\geq$ 1s
UMoments9	1164	75.436	85.5	0
BarrierData41	1320	46.910	164.6	0
Grid	4608	263.619	185.7	0
EnvelopePsiB	40	58.006	6163.3	10
ResiduesModThree	2	3.620	2274.5	2

Table 2: Decision-shaped theorem checking in five selected modules from the complete 31-artifact replay panel. Calls, CPU and maxima refer to these declarations, not all module work; $\geq$ 1s uses thread CPU.

ematical witness a low checking share is possible when source production dominates; it is not established for every mathematical module.

System-design implication. Choose the acceptance boundary before the accelerator. Mathematical witnesses motivate production/checking separation; computational certificates motivate independent kernel work. Measure the native grain before dividing it: many short declarations suggest dependency-aware declaration parallelism, while measured long calls can justify within-goal splitting. Tokens and old guarded profiles establish neither a global partition nor an accelerator ceiling.

4 Cheap checking can be hidden by imports and production coupling

RQ2: why independent verification remains costly. Source compilation imports, elaborates, executes tactics and constructs proofs before checking. A post-compilation checker can omit that production, but a new process still locates objects and prepares its environment. A potentially small kernel cost therefore need not imply cheap independent verification.

The earlier no-`-M` stock-checker point completes 15 target replays. For 13 selected short modules its process-wall median is 3.510 seconds, including lookup, imports, replay and cleanup. It is not an exclusive kernel timer or a contemporaneous construction pair. It cannot establish a current construction/replay ratio or its cause by division against an earlier, differently configured build. [**RESULT: Matched no-`-M` import-only/checking/cleanup components and a comparable independent-process baseline decomposition.**] Repeated imports and coupling are explanations to test, not a residual assigned to an unmeasured stage.

Closure (modules)	Source (s)	Check (s)	Ratio
`Nat.Basic` (1,330)	622.379	65.693	10.56%
NearPin (3,333)	2556.623	172.811	6.76%
Grid (3,793)	3878.707	563.082	14.52%
Amplitudes (5,083)	6173.195	898.034	14.55%

Table 3: One-pass no-`-M`, same-artifact closure comparisons. Source work sums compiler subprocess wall; checking is one stream's process wall. Shared dependencies are counted once within each closure, so rows are not independent trials. These are different tasks, not parallel Lake or serving speedups. Computational closures exceed the listed O-line share.

Trust scope changes the work too. Separate no-`-M` controls take 2.195 seconds import-relative and 73.134 seconds fresh-scope for Mathlib Nat.Basic, a 33.32 ratio. A NearPinDeletion fresh-scope control reaches its 180-second wall bound. These are scope contrasts, not interchangeable verification methods. A stream that checks dependencies needs a full-closure baseline.

Prototype: streamed original-kernel replay in one process. O's implementation reads module-data parts, traverses imports, retains their mapped regions and calls the upstream `Kernel.Environment.replay`. A visited set avoids processing an already visited module within the stream. Checking compiled declarations is separate from source elaboration. The implementation also has an import-relative single-module mode and malformed-proof controls. The first complete matched control covers the dependency closure of Mathlib `Data.Nat.Basic`: 1,330 modules and 132,484 declaration entries. Every module is reconstructed with the stock Lean 4.34.1 compiler, requesting `.olean`, `.ilean` and C output. The streamed checker then consumes these newly emitted objects, rather than the initially distributed cache, from one empty checked kernel environment. All commands omit `-M` and use P0–7 with external cgroup protection.

The checking process takes 65.693 seconds versus 622.379 seconds of serial source compiler work (Table 3). This is a comparison between producing and validating artifacts, not a 9.47-fold acceleration of production or serving. The numerator includes process setup, ob-

ject reads, replay and retirement; the denominator includes each module's imports, elaboration, tactics, first checking and standard artifact emission. Neither clock includes shared-lock wait or coordinator work. The streamed process consumes 65.509 user and 0.163 system CPU seconds; its `wait4` peak RSS is 754,252 KiB.

Forty-two completed single-module-oracle chunks independently reimport each target's environment before native replay; every chunk exits zero and its requested and accepted sets agree. Together they match all 1,330 streamed acceptances. A separate stock-checker preflight covers 13 modules. Three invalid proof terms—a loose variable, unknown constant and wrong sort—are rejected through both streaming and module-local replay. These checks establish differential evidence for the tested artifacts, not universal transparency or an axiom-closure audit. Upstream replay excludes 2,269 unsafe/partial entries in this closure; axioms keep their ordinary Lean semantics. The stock reader's well-formed-object assumption is retained.

The closure includes compiler and standard libraries, not only mathematical theorems. Its 80 Mathlib-namespace modules account for 0.849 seconds of kernel replay and 49.607 seconds of source compiler work. That component ratio (1.71%) omits shared-base checking, reads and process overhead; this namespace also contains tactics and helpers. It cannot replace the full-closure 10.56% process comparison or establish a mathematical-module population estimate. This one-pass, warm-cache control demonstrates a lower-cost checking boundary at complete dependency scope. It does not isolate the causal shares of import reuse and omitted production.

A computational target and its dependencies have different ratios. The reconstructed Grid closure also completes: 3,793 source modules, 3,878.707 seconds of serial compiler work, and 563.082 seconds of matched stream checking (14.52%). Its exact accepted set matches 119 completed module-oracle chunks, including 42 previously completed chunks for identical shared dependencies. The stream reports 382,707 entries and 3,554 upstream unsafe/partial exclusions. This advances the earlier observation whose module-local parity was pending. The predeclared Amplitudes and `NearPinDeletion` cases now also complete both construction and exact module-oracle acceptance parity, with 5,083 and 3,333 modules, respectively. Their matched stream/source-work fractions are 14.55% and 6.76%. All four cases use one 5,085-node source union: shared immutable dependencies are built once and charged to each containing closure. There are 161 completed oracle chunks covering those 5,085 unique modules, not four disjoint samples or repeated confidence trials.

Within that process, Grid's own kernel replay takes 255.487 seconds, versus 318.231 seconds for its source compiler invocation: 80.28%. The target replay subclock excludes shared-process startup and dependency checking; source compilation includes target imports and artifact emission. This component ratio is not standalone checking latency or a sampled CPU share. Table 4 uses a common read/map/replay subclock for the four targets. The selected AI mathematical lemma `NearPinDeletion` needs 4.47 ms at that boundary versus 0.962 seconds of source compiler wall; Amplitudes needs 110.026 versus 133.615 seconds. The contrast supports two different cost regimes on these inputs, without assigning every remaining producer millisecond to imports. The smaller whole-closure fraction includes source work for dependencies and cannot establish that a computational target itself is cheap to validate. No source-only or timing classifier for every module follows.

Target	Source (s)	Component (s)	Ratio
`Nat.Basic`	0.410718	0.026844	6.54%
NearPinDeletion	0.962274	0.004473	0.46%
Grid	318.231	255.489	80.28%
Amplitudes	133.615	110.026	82.35%

Table 4: Matched target read/map/kernel-replay components after dependency checking, versus target source compiler process wall. Components exclude shared-base checking and startup; these are not standalone latency ratios.

The actual O native cgroup limit is 11.875 GiB. Grid source compilation reaches 7,940,196 KiB peak RSS (7.57 GiB), exceeding the listed 4 GiB O-line share; the streamed closure reaches 2,328,540 KiB. This is an observed resource deviation, not a qualified 4 GiB comparison or authority to increase that share. [**RESULT: Repeated generated-library closure comparisons, causal component attribution, uncertainty and full axiom coverage.**] The complete passing-set comparisons do not establish universal shared-context transparency: duplicate theorem names, conflicting definitions and undeclared sibling visibility require separate qualification. A context-auditing continuation is not part of these timing results.

System-design implication. Expose the cheap checking boundary if one exists: separate production and checking APIs, reuse compatible contexts and charge the complete lifecycle. Warm imports are established engineering [22, 24]; the characterization must show where they matter and what remains expensive.

5 Recomputation motivates parallel original-kernel checking

RQ3: scale without adding trusted proof rules. A monolithic reduction can leave cores unavailable to one checking invocation. Parallelism requires indepen-

Arm	Kernel span (ms)	Worker CPU (ms)	Native (s)
Original	146.08	146.03	1.974
$k = 1$	144.07	144.04	5.973
$k = 2$	140.50	149.35	5.862
$k = 4$	151.68	237.61	5.897
$k = 8$	178.02	392.06	5.927

Table 5: Medians of three no-`--M` repetitions on one real Grid equality. Kernel span covers the worker checks; worker CPU sums their thread clocks. Native wall includes preparation, checks and artifact handling.

dently checkable claims and a checked connection to the unchanged original proposition. A source split, numeric evaluator or worker success flag alone is insufficient.

Prototype: generated claims and checked assembly. M freezes a real Grid `PointBox.calc` equality and structurally forms 18 scalar-equality leaves. The prototype groups them for one, two, four or eight workers, constructs decision proofs and submits declarations through the original C++ kernel's `Environment.addDeclCore` API. Checked leaves are emitted as modules; a composition proof uses extensionality lemmas and is checked against the original target. Code checks the target, sorry/metavariable absence and expected axiom boundary. The completed flat-partition diagnostic below exercises this acceptance path.

The controls distinguish original monolithic $k = 0$, the same decomposition with $k = 1$, and $k = 2, 4, 8$. This separates decomposition from parallel execution. Preparation, repeated reductions, serialization/communication, assembly, checking, memory and retirement belong in end-to-end cost. The corrected flat-partition matrix completes 15 native cells: the original and $k = 1, 2, 4, 8$, each repeated three times. All accept; final propositions and the `[propext]` axiom dependencies match the extracted original, with no `sorry` or new axiom dependency. Both original and split paths export a checked final module. Same-k repeated artifacts are byte-identical; different partitions intentionally construct different proof terms.

Table 5 shows a negative whole-path result for this partition. Relative to the same decomposition at $k = 1$, pure-kernel scaling is 1.025, 0.950 and 0.809 at $k = 2, 4, 8$; the eight-job case spends 2.72 times the worker CPU. More occupied cores do not imply useful acceleration. The split native pipeline takes roughly three times the original's wall time. At $k = 2, 4, 8$, ordinary module reimport takes median 3.643/3.659/3.660 seconds; final theorem checking takes about 0.16–0.20 ms. Environment reconstruction, rather than the small assembly proof alone, dominates the observed merge stage. This standalone goal pipeline is not a warmed service or a full Grid build.

The flat grouping places both exponential record components in one $k = 2$ job, while several $k = 8$ jobs touch the same components. This supports an imbalance/repeated-reduction hypothesis, not a complete CPU attribution. A subsequent component-preserving control also completes 15 accepting cells. Its original/$k = 1/k = 4$ kernel-span medians are 151.03/147.72/78.75 ms, showing a local reduction at this boundary. Whole native wall nevertheless rises from 1.982 seconds original to 5.838 seconds at $k = 4$. Twenty-seven same-k artifact byte comparisons differ; native inspection finds exact typed `ConstantInfo` equality for all 27 pairs, including theorem types and bodies. This is not extension-state equality or an established explanation of serialization differences. Both controls remain negative for end-to-end acceleration of this small goal, not bounds on other decompositions or larger goals.

Both partition points' recorded PID/birth identities are retired. Their recorded resource response is a 14 GiB sampled tree-RSS stop with a 180-second self alarm. Their frozen runners and receipts do not establish a dedicated charged-memory cgroup, so these diagnostics are not presented as the required final cgroup-controlled resource comparison. No observed cell times out or reaches its sampled stop. Aggregate heartbeat equivalence to the monolithic request is not implemented. [**RESULT: Final resource-qualified parallel comparison, corrupt-leaf and budget/failure equivalence, shape coverage and heterogeneous-module transfer.**] One frozen equality is not full Grid or a general extraction algorithm.

Declaration-level replay on a complete module. The measured subsecond calls motivate a second prototype: dependency-aware replay of the original declarations, without rewriting their proof terms. Barrier-Data41 has 3,425 supported safe, acyclic declarations. The prototype creates ready frontiers, checks jobs with the stock kernel, exports checked module fragments and normally imports them for successor frontiers. It explicitly rejects unsupported or cyclic input. The final native audit compares every typed `ConstantInfo`, including theorem bodies, and returns names in the original array order.

All 15 cells accept the same 3,425 typed constants and ordered names; all 99 native stages exit zero and their recorded identities/cgroups retire. Eight workers reduce the measured pipeline median from 46.992 to 14.127 seconds, 3.326-fold against the direct stock-API baseline (Table 6); per-repetition ratios range from 3.284 to 3.359. Native CPU increases 27.8%. One-worker wave replay is 13.5% slower than direct stock replay, so a wave-one-only baseline would overstate the gain. The point uses P0–7, no `-M`, and an external 14 GiB charged-memory cap.

The output contract is the final typed-constant/name audit. Intermediate serialization is a cost of this implementation, not an extra required output imposed on the direct baseline. Original proof terms and the original kernel are retained under this local checked-fragment

Replay path	Pipeline wall (s)	Native CPU (s)
Direct stock API	46.992	46.956
Wave, 1 worker	53.321	53.205
Wave, 2 workers	31.072	54.556
Wave, 4 workers	20.233	58.335
Wave, 8 workers	14.127	60.016

Table 6: BarrierData41 independent compiled-module replay, three ordered repetitions per path. Wall spans first stage-configuration write through final audit retirement, including graph planning, native processes, imports, exports and driver gaps. Lease wait and initial directory/environment setup are excluded; CPU sums native processes, excluding the Python coordinator.

Path	Kernel span (s)	Runtime (s)	CPU (s)
Original	6.125	9.848	9.836
1 chunk	6.177	14.347	14.319
2 chunks	3.531	11.490	14.892
4 chunks	1.967	10.331	15.307
8 chunks	0.973	8.982	15.110

Table 7: EnvelopePsiB helper, medians of three repetitions. Kernel span runs from the first worker's stock checking-call entry to the last exit. Runtime includes both sequential producer and merger process lifecycles, imports, exports and interphase cleanup. Native CPU excludes the Python parent; transformation generation, base/merge-lemma compilation and the original library producer build are excluded.

protocol; this is not a proof for arbitrary coordinators, untrusted object transfer or changed failure/budget semantics. The import-relative dependency trust matches the stock module API. Upstream checking already dispatches independent modules as tasks [12]; this case isolates parallelism inside one module, without establishing whole-library or source-elaboration speedup. Other modules, warm-service throughput and repeated failure behavior remain [**RESULT: unmeasured at the matched resource/trust boundary**].

Parallel leaves can lose their gain at the module boundary. A larger measured helper, `envelopeExpB16_calc` in EnvelopePsiB, checks 31 finite-index interval-exponential equalities. The prepared transformation groups indices into 1, 2, 4 or 8 chunks, retaining each index's original computation. An original-kernel-checked merge lemma reconstructs the original target; an explicit equality control connects that target to the released helper. This case tests parallel rechecking of a prepared transformation, not the cost of automatically producing that transformation.

Eight chunks shorten the kernel span 6.292-fold, but runtime decreases only from 9.848 to 8.982 seconds: 1.096-fold, or 8.79% less wall time. Native CPU rises 53.6%. Paired runtime ratios are 1.096, 1.103 and 0.949; the last repetition is 5.35% slower than its original. Three repetitions, with reversed setting order in the second, do not establish population uncertainty or guaranteed nonregression. At eight chunks, merger import alone takes a median 3.142 seconds, whereas final kernel assembly takes 0.296 ms. The paired nonkernel residual added over the original is 4.24–4.91 seconds; this residual includes multiple operations and is not a causal attribution to imports alone.

All 15 performance cells accept the same extracted target and the exact axiom set `[propext]`, without `sorry` or added axioms. A false-decision control reaches explicit original-kernel rejection. The 28 recorded native phases retire with their cgroups, with P0–7, no `-M` and an external 14 GiB charged-memory cap. Sampled own-tree RSS peaks at 9.087 GB and charged memory at 2.588 GB; these are distinct measurements, not equivalent limits. All 84 same-setting repeat artifact comparisons are recorded equal; proof artifacts intentionally differ across chunk counts. Aggregate heartbeat-budget transparency and arbitrary tactic-backtracking parity remain unimplemented. This helper's small and variable runtime gain complements the Barrier module case; their different baseline and output interfaces prevent combining the speedup ratios into a library estimate. It also identifies a concrete design cost: preserve checking parallelism without rebuilding a prepared environment at every assembly boundary.

A cluster cannot silently trust remote object files or worker verdicts as new proof assertions. Specify what the acceptance authority checks, dependency identity, transfer and recovery. [**RESULT: Multi-machine protocol, native soundness/differential evidence and complete cluster costs.**] Single-host code and multicore timings are not distributed results.

Earlier certificate proofs establish selected feasibility; seven Int/list original/certificate pairs all accept, but currently matched transfer costs remain unestablished. Changing a certified algorithm and parallelizing a fixed decomposition are distinct. Reflection and proof parallelism are established approaches [6, 8, 5].

System-design implication. Expose work at a kernel-checkable boundary; retain original checking rules and the required axiom closure. Keep generation/scheduling outside the trusted decision. Preserve shared subcomputations when choosing work units and account for environment reconstruction. Validate decomposition and parallelism separately, then check the complete assembled theorem at equal CPU/memory resources. Faster leaves or transformed subgoal success alone are insufficient.

6 Verified leaves build, outcome and axiom boundaries open

RQ4: what has actually been established? The publisher's default root reaches 130,834 of 163,097 modules. The other 32,263 contain 451,008 of 1,354,379 masked kernel-decision sites, 33.30%. This is not an omitted CPU fraction or a statement that outside files fail. All 562 PlanarPacking and 466 Transonic modules are outside. A successful default build cannot establish all released-artifact coverage.

Source sites, declarations and axioms differ. In the retained 363-object census, 45,661 decision-shaped auxiliaries have 42,610 globally distinct expression-type keys. Source occurrences can expand into several helpers; imported objects use another boundary. Neither these keys nor Lean's branch-local auxiliary cache establishes safe cross-proof reuse [11, 13]. Zero direct native-evaluation tokens does not establish zero transitive axiom dependence. Vericoding source flags and publisher labels similarly do not establish native acceptance or known axiom closure. [**RESULT: Repository-wide declaration reachability, admissions and axiom closure.**]

A shared context can change which proof is checked. A later combined streaming prototype exposes a concrete context failure. A valid earlier sibling provides `Shared.t : True`. A different incoming module imports only `Init.Prelude`; its raw declarations give `Shared.helper : True := Shared.t` and `Shared.t : True := Shared.helper`. These are deliberately cyclic proof data, not successfully compiled source. Version 4 accepts the incoming module after the sibling: the old `Shared.t` makes the helper check, masking the incoming local cycle. The same binary's isolated module-local original-kernel path rejects it with an unknown-constant error for `Shared.t`.

This is a demonstrated acceptance-contract mismatch, not an original-kernel bug or an accepted proof of `False`: both claimed propositions are `True`. The defect is substituting an out-of-import binding while validating the incoming proof. The repaired prototype follows incoming local dependencies transitively; a cycle takes module-local replay. It rejects this input with one and eight workers, and completes 20 native boundary controls, including sibling visibility, corrupted duplicate proofs, conflicting definitions and stale/duplicate checked tokens. All 22 recorded control processes, including historical acceptance and the isolated rejection, retire without resource interruption. These controls bound the tested cases, not arbitrary corrupted modules. The previous 18-cell, 1,330-module positive-input performance table belongs to the defective version and is not a repaired-version throughput result. [**RESULT: Complete repaired-version paired performance and broader module-context differential evidence.**]

A second boundary is implementation trust. This combined in-memory prototype commits checked results through a checked-token coordinator and a pinned internal-ABI C map-insertion adapter. Those components are additional trusted implementation code, although the original kernel still checks mathematical declarations. They differ from the earlier Barrier checked-fragment/normal-import protocol. Neither prototype establishes a universally unchanged implementation TCB.

Settings and observers change outcomes. Six released EPFL exact-program controls fail at recursion depth128 but pass at256/default512 in reconstructed contexts; producer context is unknown. A historical `-M8192` diagnostic changes seven error flags among 200 profiler/control pairs. That is neither full-source accepted-set loss nor a current no-`-M` observer-cost estimate. A C-timing control preserves diagnostics but changes allocation-heartbeat deltas; equal error flags do not establish exact budget accounting. [**RESULT: No-`-M` repeated observer/outcome stability and software coverage.**]

The retained candidate contrasts have different failure/cost boundaries. One miniF2F point spends73.21% of summed physical worker-attempt time on budget-source attempts, not that share of concurrent makespan. ProofNet pass1 has an11.57% budget-source request-time share. AutoVerus's slowest1.6% consumes about3.4%, a slow-tail rather than budget-failure metric. These historical resource points aren't a regime classifier for the library. Ordinary errors and resource exhaustion do not establish falsehood.

System-design implication. Report build domain, checked constants, trusted imports, axiom/admission policy, executable/options, resource accounting and observer configuration. A Boolean result hides these boundaries. Faster delivery of a narrower checking scope isn't equivalent verification.

7 Limits and unfinished evidence

The complete lexical comparison establishes concentration, not runtime prevalence or a universal two-way partition. The no-`-M` sample remains incomplete and purposeful modules receive no population weights. [**RESULT: Current per-regime profiles, expansion estimates and uncertainty.**] Generated-software contexts/provenance differ; software-stage costs remain [**RESULT: pending**].

The prototypes have bounded acceptance and cost observations, rather than general accepted-set transparency, soundness or cluster results. Broader differentials, axiom closure, failures and resource-qualified life-

cycle comparisons remain open. Ordinary warm-state reuse and frozen arithmetic oracles are prior mechanisms, not the contribution. The historical GPU screen was slower and removed no timeout; that does not rule out all devices or workloads. All failed, partial and withdrawn evidence remains, including interrupted state-reuse records whose missing entries aren't zero cache opportunity.

8 Related work

Proof-assistant runtime and build performance. Performance engineering of proof assistants predates AI-generated proofs. Lean's architecture and reference-counting work study efficient execution of functional metaprograms [3, 30]. Tebbi and Gross's Ltac profiler attributes nested tactic invocations in Coq [29]. Huch and Wenzel study distributed Isabelle/AFP builds and schedules for multithreaded sessions [5]. These establish runtime, tactic and dependency-scheduling costs as important problems. Our work measures which of these boundaries dominate particular generated artifacts; low realized core use does not by itself establish a new scheduling or runtime technique.

Mathlib compilation and continuous integration. Community profiling has already examined per-file build times and typeclass, tactic, import and checking phases in Mathlib [28]. We do not claim the first library cost profile. Mathlib's official cache downloads compiled artifacts keyed by toolchain, source and imported dependencies; CI constructs and publishes those artifacts [19]. This makes incremental CI, cached downstream builds and fresh source compilation different cost boundaries. Prior Isabelle-build work also discusses Mathlib's cached workflow in its contemporary setting [5]. We do not import its CI duration or community profiler percentages into our hardware/version denominators. Our new-source study distinguishes default-root coverage, successful artifact growth, CPU/system time and uncompleted probability draws rather than treating a cache hit or a build command's completion as all-source coverage.

Native replay and external proof checkers. The archived `lean4checker` is now distributed with Lean as `leanchecker`. It replays declarations through the official kernel: it is a separate checking pass, not a separately implemented kernel [16]. Carneiro's Lean4Lean reimplements the checker in Lean and evaluates post-compilation replay on major libraries [2]. Lean Kernel Arena evaluates multiple implementations on exported environments, including acceptance, rejection and corner cases [15]. Its instruction-derived virtual CPU scores and a local compiler wall clock have different meanings. These studies address checking performance and trust at post-export or compiled-environment boundaries. Our measurements additionally charge source elaboration, imports, emitted artifacts and failed candidate work; a faster external kernel is useful only for the share at its boundary. Axioms, admissions and native evaluation remain separate trust questions.

RL verification services and prepared proof state. Kimina Lean Server uses concurrent persistent REPLs and cached import headers for RL and proof feedback [22, 24]. Its published throughput study establishes import reuse as a practical service technique, not a new mechanism here. Snapshots retain live proof state for tactic search [26]; stock incremental elaboration reuses prior source state [10]. APE-Bench evaluates file/repository-level proof engineering with pinned library versions and shared infrastructure [32]. Such services and benchmarks establish why reusable contexts matter. Our comparison asks where resource-bounded time and memory go across research mathematics, software artifacts and candidate streams, and whether profiling or changed limits alter their measured verdicts. We do not transfer service speedups to a serial library build.

Reuse, scheduling and certified computation. Fine-grained verification caching [17], tactic/prefix reuse [1, 27], simplifier memoization [9], dynamic dependencies [23, 4] and age-based scheduling [18, 25] are established alternatives. The streaming and chunking prototypes test different regime-specific implications; external search with native checking is prior art.

The gap addressed here is a boundary-consistent, two-regime characterization of released AI-generated formal artifacts, with a research-level library as the primary population and rejected/resource-limited attempts retained as contrasts. It joins source/build coverage to explicitly scoped native costs, treats mapped resident pages and mutable state as distinct metrics, and tests whether observation preserves outcomes. It complements build engineering and checker benchmarks rather than claiming that their bottlenecks are new. Complete software and causal-profile findings remain explicit open results.

9 Conclusion

AI-generated mathematics exposes two verification boundaries: checking mathematical witnesses and recomputing proof-embedded calculations. The matched lexical census shows a large increase concentrated in a small source subset; its full-library time weight remains unknown. Decoupled streaming and original-kernel chunking test different systems responses. [**RESULT: Final no-`-M` regime weights, matched independent-checking costs and whole-path parallel/cluster results.**] Build coverage, reproducibility and axiom

closure must accompany every performance or verified claim.

References

[1] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. The tactician (extended version): A seamless, interactive tactic learner and prover for coq, 2020. Extended version of the CICM paper.

[2] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 156–166, 2014.

[5] Fabian Huch and Makarius Wenzel. Distributed parallel build for the isabelle archive of formal proofs. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPIcs*, pages 22:1–22:19, 2024.

[6] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[7] Lean community. Mathlib4 reference source at d13f23b7. https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612/Mathlib, 2026. Pinned Mathlib subtree; local Git revision and unchanged source checked 1 November 2026. Lexical census excludes comments and strings.

[8] Lean contributors. Lean 4.9.0-rc1 ordinary decision tactic implementation. Pinned evalDecide handler; accessed 30 October 2026.

[9] Lean contributors. Lean4.22.0 release: simplifier cache reuse, 2025. Simp section; PR 8880.

[10] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[11] Lean contributors. Lean 4.34.1 auxiliary-lemma implementation, 2026. Branch-local cache lookup; accessed 30 October 2026.

[12] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026. Pinned SDK source inspected 1 November 2026; independently dispatched module tasks, not a measured whole-library baseline.

[13] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026. doKernel invokes mkAuxLemma; accessed 30 October 2026.

[14] Lean contributors. Lean 4.34.1 runtime memory-budget checks, 2026. Source and matching SDK disassembly inspected 31 October 2026.

[15] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[16] Lean contributors. lean4checker and the built-in leanchecker, 2026. Official archived README; integrated into Lean from v4.28.0; accessed 31 October 2026.

[17] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[18] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[19] Mathlib contributors. Mathlib build cache and ci uploads, 2026. Official cache README and Cache/CI.md, accessed 31 October 2026; general CI context, not measured CI cost.

[20] Meta Platforms. Lean Kernel Verify: Compiler-based safety checker for gpu kernels, 2026. Pinned release 129e65bfd8efbf6c45710732d205c941ef65928c; accessed 30 October 2026.

[21] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[22] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[23] Salsa contributors. The red-green algorithm, 2026. Official algorithm documentation; accessed 30 October 2026.

[24] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean

server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[25] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. arXiv:1712.00790v2, 2018.

[26] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[27] SHRED contributors. SHRED: Reuse repeated work across thousands of Lean proof attempts, 2026. Software and experimental artifact; revision a6616e99534ce777dcebfd56566eaaffb63997a8; accessed 16 October 2026.

[28] Michael Stoll. Ways to speed up mathlib. Lean community Zulip archive, 25 April 2024, 2024. Community measurement report; no timing or phase percentage transferred to our experiments.

[29] Tobias Tebbi and Jason Gross. A profiler for ltac. CoqPL Workshop abstract, 2015.

[30] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Proceedings of the International Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2020.

[31] Vericoding contributors. Vericoding: released model-tagged programs and final experiment tables. Artifact repository, revision b265a3b2bce31e09d264306f419e810d02f98f44, 2026. Inspected 31 October 2026; final outputs do not constitute complete verification trajectories.

[32] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. Ape-bench: Evaluating automated proof engineering for formal math libraries, 2026. Version 3, revised 2 February 2026.

[33] Zhe Ye, Hantao Lou, Yuechun Sun, et al. Vero: Can ai agents build formally verified software repositories?, 2026. Official repository inspected at 0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09; evaluated run outputs not established.